CREATOR

AI ASSISTANT

Final Version — All Features Included

Developer: Aditya Chauhan | Python 3.9.11 | May 2026

SECTION 1 — COMPLETE CHANGELOG

Every change made to CREATOR from V1 to Final Version

Version	Changes
V1	Basic voice assistant — text input only
V2	Added Whisper voice recognition + pyttsx3 TTS
V3	Added GUI + multi-command + file control + image gen
V4	Jarvis GUI + arc reactor + manual text input + base model
V4.1	Hindi + English bilingual support
V4.2	Selenium + EdgeDriver + WhatsApp messaging
V4.3	Smart search opens Edge instead of speaking
V4.4	WhatsApp + Instagram open as desktop apps
V4.5	Master → Sir + push-to-listen only + 3s detection
Final	All fixes combined — production ready

SECTION 2 — COMPLETE SETUP GUIDE

2.1 Install Libraries

```
pip install numpy sounddevice psutil keyboard requests pyttsx3 openai openai-whisper selenium
```

2.2 Download Whisper Model

```
python -c "import whisper; whisper.load_model('base')"
```

2.3 Install ffmpeg

```
winget install ffmpeg
```

2.4 EdgeDriver Setup

- Download from: https://developer.microsoft.com/en-us/microsoft-edge/tools/webdriver/
- Copy msedgedriver.exe to your CREATOR(PROJECT1) folder

2.5 Run CREATOR

```
cd "C:\Users\MY PC\CREATOR(PROJECT1)"
python creator.py
```

SECTION 3 — ALL COMMANDS REFERENCE

3.1 Activation

Action	Result
Press Ctrl+Space → say CREATOR	YES SIR popup + active mode
Click ACTIVATE button	Same
Say: bye creator	GOODBYE SIR animation + standby
Say: cancel / cross	Stops current task

3.2 Apps & Websites

Command	Opens
open whatsapp	WhatsApp desktop app
open instagram	Instagram desktop app
open youtube / gmail / spotify	Browser
open chrome / edge / brave	Browser app
open notepad / calculator / paint	Windows app
open vscode / cmd / task manager	Windows app
open edge and go to youtube.com	Edge → YouTube
search on edge for python	Edge → Bing search

3.3 System Controls

Command	Result
set brightness to 70	Screen brightness 70%
turn wifi on / off	WiFi toggle
turn bluetooth on / off	Bluetooth toggle
shutdown / restart / sleep	PC power control
system info / status	CPU + RAM + Battery

3.4 File Control

Command	Result
create file notes.txt	Creates file
create file notes.txt with content Hello	Creates with content
read file notes.txt	Reads aloud

delete file notes.txt	Deletes with permission
add text Hello to notes.txt	Appends text
rewrite notes.txt with content New	Overwrites
rename file old.txt to new.txt	Renames
create / delete folder MyProject	Folder control
list files in Documents	Lists files

3.5 Smart Search + WhatsApp

Command	Result
what is python	Opens Edge with Google results
who is Elon Musk	Opens Edge with Google results
how to make pasta	Opens Edge with Google results
send whatsapp message to Aditya hello	Sends WhatsApp message

3.6 Hindi Commands

Hindi	English Equivalent
क्रोम खोलो	open chrome
यूट्यूब खोलो	open youtube
बंद करो	bye creator
सिस्टम जानकारी	system info
बंद करो कंप्यूटर	shutdown

SECTION 4 — FINAL COMPLETE CODE

Save as creator.py in CREATOR(PROJECT1) folder. Run: python creator.py

Total lines: 1404 | Version: Final | May 2026

```
# ======================================================================
# CREATOR AI ASSISTANT — VERSION 4.0
# Jarvis-Style GUI + Manual Control + Voice + Base Model (Faster)
# Developer: Aditya Chauhan | Python 3.9.11 | May 2026
# Run as Administrator for OS controls
# ======================================================================

import os, re, time, wave, threading, sys, shutil, math
import subprocess, webbrowser, urllib.request
from selenium import webdriver
from selenium.webdriver.edge.service import Service
from selenium.webdriver.edge.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import tkinter as tk
from tkinter import scrolledtext
import numpy as np
import sounddevice as sd
import psutil, whisper, pyttsx3, keyboard
from datetime import datetime

# ============================================================
# SECTION 1 — CONFIGURATION
# ============================================================
WAKE_WORD           = "creator"
END_WORDS           = ["bye creator"]
CANCEL_WORDS        = ["cancel", "cross"]
SHORTCUT_KEY        = "ctrl+space"
LISTEN_DURATION     = 6
WAKE_CHECK_DURATION = 3            # Longer — better detection
SAMPLE_RATE         = 16000
MODEL_SIZE          = "base"       # Multilingual — Hindi + English
TTS_RATE            = 165
AUDIO_FILE          = "input.wav"
EDGE_PATH      = r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe"
EDGE_DRIVER    = r"C:\Users\MY PC\CREATOR(PROJECT1)\msedgedriver.exe"

def get_edge_driver(headless=False):
    """Returns a Selenium Edge driver instance."""
    opts = Options()
    if headless:
        opts.add_argument("--headless")
    opts.add_argument("--start-maximized")
    svc = Service(EDGE_DRIVER)
    return webdriver.Edge(service=svc, options=opts)

def send_whatsapp_message(contact, message):
    """Opens WhatsApp Web and sends message to contact."""
    if not ask_permission(f"send WhatsApp message to {contact}"):
        speak("Cancelled."); return
    speak(f"Opening WhatsApp to message {contact}.")
    try:
        url = f"https://web.whatsapp.com/send?phone=&searchTerm={contact}"
        driver = get_edge_driver()
        driver.get("https://web.whatsapp.com")
        speak("Please scan WhatsApp QR code if asked. Say yes when ready.")
        record_audio(10)
        resp = transcribe_audio()
        if "yes" not in (resp or "").lower():
            speak("Cancelled."); driver.quit(); return
        driver.get(url)
        time.sleep(4)
        search = WebDriverWait(driver, 15).until(
            EC.presence_of_element_located(
```

```
                (By.XPATH, '//div[@contenteditable="true"][@data-tab="3"]')))
        search.click(); search.send_keys(contact)
        time.sleep(2)
        first = WebDriverWait(driver, 10).until(
            EC.presence_of_element_located(
                (By.XPATH, '//div[@role="listitem"]')))
        first.click(); time.sleep(2)
        msg_box = WebDriverWait(driver, 10).until(
            EC.presence_of_element_located(
                (By.XPATH, '//div[@contenteditable="true"][@data-tab="10"]')))
        msg_box.click(); msg_box.send_keys(message)
        from selenium.webdriver.common.keys import Keys
        msg_box.send_keys(Keys.ENTER)
        speak(f"Message sent to {contact}.")
        log_gui(f"WhatsApp: '{message}' sent to {contact}", "system")
        time.sleep(2); driver.quit()
    except Exception as e:
        speak("Could not send WhatsApp message.")
        log_gui(f"WhatsApp error: {e}", "error")

def smart_search(query):
    """Opens Edge and searches — shows results on screen."""
    speak(f"Searching for {query}.")
    url = f"https://www.google.com/search?q={query.replace(' ', '+')}"
    subprocess.Popen([EDGE_PATH, url])
    log_gui(f"Search opened: {query}", "system")

STANDBY = "STANDBY"
ACTIVE  = "ACTIVE"

# ── JARVIS COLOR PALETTE ──────────────────────────────────────
BG_DEEP   = "#000B1A"
BG_MID    = "#001428"
BG_PANEL  = "#001E3C"
BG_ENTRY  = "#00111F"
CYAN      = "#00D4FF"
CYAN_DIM  = "#0077AA"
BLUE      = "#0055AA"
GREEN     = "#00FF88"
GREEN_DIM = "#006633"
RED       = "#FF3355"
ORANGE    = "#FF8C00"
TEXT_PRI  = "#B8D4F0"
TEXT_DIM  = "#2A5A8A"
TEXT_LOG  = "#7AB8E0"
GOLD      = "#FFD060"
WHITE     = "#FFFFFF"

# ── FONTS ─────────────────────────────────────────────────────
FT = "Consolas"

# ═══════════════════════════════════════════════════════════════
# SECTION 2 — STATE MANAGER
# ═══════════════════════════════════════════════════════════════
class AppState:
    def __init__(self):
        self.mode     = STANDBY
        self.lock     = threading.Lock()
        self.cancel   = threading.Event()
        self.shutdown = threading.Event()

    def set_mode(self, m):
        with self.lock: self.mode = m

    def get_mode(self):
        with self.lock: return self.mode

app = AppState()

# ═══════════════════════════════════════════════════════════════
# SECTION 3 — OPENAI SETUP
# ═══════════════════════════════════════════════════════════════
API_KEY = os.getenv("OPENAI_API_KEY")
AI_AVAILABLE = False
client = None
```

```
if API_KEY:
    try:
        from openai import OpenAI
        client = OpenAI(api_key=API_KEY)
        AI_AVAILABLE = True
        print("[INFO] AI mode ENABLED.")
    except Exception as e:
        print(f"[WARNING] OpenAI failed: {e}")
else:
    print("[INFO] Command-Only mode — no API key.")

# ═══════════════════════════════════════════════════════════════
# SECTION 4 — WHISPER (base model — faster)
# ═══════════════════════════════════════════════════════════════
print(f"[INFO] Loading Whisper '{MODEL_SIZE}' model...")
whisper_model = whisper.load_model(MODEL_SIZE)
print("[INFO] Whisper ready.")

# ═══════════════════════════════════════════════════════════════
# SECTION 5 — TEXT TO SPEECH
# ═══════════════════════════════════════════════════════════════
tts_engine = pyttsx3.init()
tts_engine.setProperty("rate", TTS_RATE)
tts_lock = threading.Lock()

def speak(text):
    print(f"CREATOR: {text}")
    log_gui(f"CREATOR: {text}", "response")
    try:
        sd.stop()
        with tts_lock:
            tts_engine.say(text)
            tts_engine.runAndWait()
        time.sleep(0.3)
    except Exception as e:
        print(f"[TTS ERROR] {e}")

# ═══════════════════════════════════════════════════════════════
# SECTION 6 — AUDIO
# ═══════════════════════════════════════════════════════════════
def save_wav(fn, audio, sr=SAMPLE_RATE):
    a = np.int16(np.clip(audio, -1.0, 1.0) * 32767)
    with wave.open(fn, "wb") as wf:
        wf.setnchannels(1); wf.setsampwidth(2)
        wf.setframerate(sr); wf.writeframes(a.tobytes())

def record_audio(dur, fn=AUDIO_FILE, sr=SAMPLE_RATE):
    try:
        au = sd.rec(int(dur*sr), samplerate=sr, channels=1, dtype="float32")
        sd.wait(); save_wav(fn, au.flatten(), sr); return True
    except Exception as e:
        print(f"[MIC ERROR] {e}"); return False

def transcribe_audio(fn=AUDIO_FILE):
    try:
        r    = whisper_model.transcribe(fn, task="transcribe")
        lang = r.get("language", "en")
        text = (r.get("text") or "").strip()
        print(f"[LANG: {lang.upper()}] {text}")
        return text
    except Exception as e:
        print(f"[WHISPER ERROR] {e}"); return ""

# ═══════════════════════════════════════════════════════════════
# SECTION 7 — JARVIS GUI COMMAND CENTER
# ═══════════════════════════════════════════════════════════════
gui_root   = None
status_var = None
mode_var   = None
log_widget = None
canvas_ani = None
entry_var  = None
gui_ready  = threading.Event()

# Animation state
```

```
ani = {
    "angle"    : 0,
    "pulse"    : 0,
    "pulse_dir": 1,
    "ring_r"   : [50, 80, 110],
    "ring_max" : [90, 120, 150],
    "ring_min" : [50, 80, 110],
    "active"   : False
}

def draw_hud_corners(c, w, h, color=CYAN, size=24, thick=2):
    """Draw HUD-style corner brackets."""
    s = size
    c.create_line(0,s,0,0,    fill=color, width=thick)
    c.create_line(0,0,s,0,    fill=color, width=thick)
    c.create_line(w-s,0,w,0,  fill=color, width=thick)
    c.create_line(w,0,w,s,    fill=color, width=thick)
    c.create_line(0,h-s,0,h,  fill=color, width=thick)
    c.create_line(0,h,s,h,    fill=color, width=thick)
    c.create_line(w-s,h,w,h,  fill=color, width=thick)
    c.create_line(w,h-s,w,h,  fill=color, width=thick)

def animate_reactor():
    """Continuously animate the arc reactor canvas in main GUI."""
    global canvas_ani
    if not canvas_ani or not gui_root:
        return

    c  = canvas_ani
    cx = 130; cy = 130
    c.delete("anim")

    is_active = app.get_mode() == ACTIVE
    speed  = 10 if is_active else 4
    pspeed = 3  if is_active else 1

    ani["angle"] = (ani["angle"] + speed) % 360
    ani["pulse"] += pspeed * ani["pulse_dir"]
    if ani["pulse"] >= 30 or ani["pulse"] <= 0:
        ani["pulse_dir"] *= -1

    pulse      = ani["pulse"]
    main_color = GREEN if is_active else CYAN
    dim_color  = GREEN_DIM if is_active else CYAN_DIM

    # ── Outer dashed ring ──────────────────────────
    for i in range(0, 360, 20):
        a1 = math.radians(i + ani["angle"] % 20)
        a2 = math.radians(i + 12 + ani["angle"] % 20)
        r  = 120 + pulse * 0.15
        x1 = cx + r * math.cos(a1); y1 = cy - r * math.sin(a1)
        x2 = cx + r * math.cos(a2); y2 = cy - r * math.sin(a2)
        c.create_line(x1, y1, x2, y2, fill=dim_color,
                      width=1, tags="anim")

    # ── Middle ring ────────────────────────────────
    r2 = 85 + pulse * 0.2
    c.create_oval(cx-r2, cy-r2, cx+r2, cy+r2,
                  outline=main_color, width=1, tags="anim")

    # ── Scanner arc ────────────────────────────────
    sa = ani["angle"]
    c.create_arc(cx-95, cy-95, cx+95, cy+95,
                 start=sa, extent=70,
                 outline=main_color, width=2,
                 style="arc", tags="anim")
    # Scanner trail
    c.create_arc(cx-95, cy-95, cx+95, cy+95,
                 start=sa+70, extent=40,
                 outline=dim_color, width=1,
                 style="arc", tags="anim")

    # ── Inner circle ───────────────────────────────
    r3 = 44 + pulse * 0.1
    c.create_oval(cx-r3, cy-r3, cx+r3, cy+r3,
```

```
                  outline=main_color, width=2, tags="anim")
    c.create_oval(cx-r3+6, cy-r3+6, cx+r3-6, cy+r3-6,
                  outline=dim_color, width=1, tags="anim")

    # ── Center dot ──────────────────────────────────────
    c.create_oval(cx-6, cy-6, cx+6, cy+6,
                  fill=main_color, outline="", tags="anim")

    # ── Cross hairs ─────────────────────────────────────
    c.create_line(cx-40, cy, cx-50, cy,
                  fill=dim_color, width=1, tags="anim")
    c.create_line(cx+40, cy, cx+50, cy,
                  fill=dim_color, width=1, tags="anim")
    c.create_line(cx, cy-40, cx, cy-50,
                  fill=dim_color, width=1, tags="anim")
    c.create_line(cx, cy+40, cx, cy+50,
                  fill=dim_color, width=1, tags="anim")

    # ── Mode text ───────────────────────────────────────
    mode_txt = "ACTIVE" if is_active else "STANDBY"
    c.create_text(cx, cy, text=mode_txt,
                  font=(FT, 11, "bold"),
                  fill=main_color, tags="anim")
    c.create_text(cx, cy+18,
                  text="◈ ONLINE ◈" if is_active else "◈ READY ◈",
                  font=(FT, 7), fill=dim_color, tags="anim")

    if gui_root:
        gui_root.after(40, animate_reactor)

def update_stats():
    """Update CPU/RAM stats in GUI every 3 seconds."""
    if not gui_root: return
    try:
        cpu  = psutil.cpu_percent(interval=None)
        ram  = psutil.virtual_memory().percent
        bat  = psutil.sensors_battery()
        batt = f"{int(bat.percent)}%" if bat else "N/A"
        if status_var:
            mode = app.get_mode()
            if mode == ACTIVE:
                status_var.set(
                    f"CPU {cpu:.0f}%   RAM {ram:.0f}%   "
                    f"BAT {batt}   ◈ LISTENING")
            else:
                status_var.set(
                    f"CPU {cpu:.0f}%   RAM {ram:.0f}%   "
                    f"BAT {batt}   ◈ STANDING BY")
    except Exception:
        pass
    if gui_root:
        gui_root.after(3000, update_stats)

def build_gui():
    global gui_root, status_var, mode_var, log_widget
    global canvas_ani, entry_var

    gui_root = tk.Tk()
    gui_root.title("CREATOR  ◈  AI ASSISTANT")
    gui_root.geometry("860x640")
    gui_root.configure(bg=BG_DEEP)
    gui_root.resizable(True, True)
    gui_root.minsize(700, 500)

    # ── TOP BAR ─────────────────────────────────────────
    top = tk.Frame(gui_root, bg=BG_MID, height=54)
    top.pack(fill="x")
    top.pack_propagate(False)

    # HUD corners on top bar via canvas
    top_c = tk.Canvas(top, bg=BG_MID, height=54,
                      highlightthickness=0)
    top_c.pack(fill="x")
    top_c.update_idletasks()
```

```
    def draw_top(event=None):
        w = top_c.winfo_width()
        top_c.delete("all")
        # Title
        top_c.create_text(w//2, 27,
                          text="◈  C · R · E · A · T · O · R  ◈",
                          font=(FT, 16, "bold"), fill=CYAN)
        top_c.create_text(w//2, 44,
                          text="A D V A N C E D  A I  A S S I S T A N T",
                          font=(FT, 7), fill=CYAN_DIM)
        # Corner lines
        top_c.create_line(0, 10, 0, 0, fill=CYAN, width=2)
        top_c.create_line(0, 0, 60, 0, fill=CYAN, width=2)
        top_c.create_line(w-60, 0, w, 0, fill=CYAN, width=2)
        top_c.create_line(w, 0, w, 10, fill=CYAN, width=2)
        top_c.create_line(0, 53, 0, 44, fill=CYAN, width=2)
        top_c.create_line(w, 53, w, 44, fill=CYAN, width=2)
        # Decorative lines
        top_c.create_line(10, 53, w-10, 53, fill=CYAN_DIM, width=1)

    top_c.bind("<Configure>", draw_top)

    # ── STATUS BAR ──────────────────────────────────────
    stat_f = tk.Frame(gui_root, bg=BG_MID, height=28)
    stat_f.pack(fill="x")
    stat_f.pack_propagate(False)

    status_var = tk.StringVar(value="CPU --%   RAM --%   BAT --%   ◈ STANDING BY")
    tk.Label(stat_f, textvariable=status_var,
             font=(FT, 9), fg=CYAN_DIM, bg=BG_MID,
             anchor="w", padx=12).pack(fill="x")

    # ── MAIN AREA ───────────────────────────────────────
    main_f = tk.Frame(gui_root, bg=BG_DEEP)
    main_f.pack(fill="both", expand=True, padx=8, pady=4)

    # Left panel — Arc Reactor
    left = tk.Frame(main_f, bg=BG_DEEP, width=270)
    left.pack(side="left", fill="y", padx=(0,6))
    left.pack_propagate(False)

    tk.Label(left, text="◈  SYSTEM CORE",
             font=(FT, 8), fg=TEXT_DIM,
             bg=BG_DEEP).pack(anchor="w", pady=(4,0))

    canvas_ani = tk.Canvas(left, width=260, height=260,
                           bg=BG_MID, highlightthickness=1,
                           highlightbackground=CYAN_DIM)
    canvas_ani.pack(pady=4)
    draw_hud_corners(canvas_ani, 260, 260, CYAN_DIM, 18, 1)

    # Mode label below reactor
    mode_var = tk.StringVar(value="◉  STANDBY")
    mode_lbl = tk.Label(left, textvariable=mode_var,
                        font=(FT, 13, "bold"),
                        fg=CYAN, bg=BG_DEEP)
    mode_lbl.pack(pady=(2,0))

    # Divider
    tk.Frame(left, bg=CYAN_DIM, height=1).pack(fill="x", pady=4)

    # Quick commands panel
    tk.Label(left, text="◈  QUICK COMMANDS",
             font=(FT, 8), fg=TEXT_DIM,
             bg=BG_DEEP).pack(anchor="w")

    quick_cmds = [
        ("system info",    "SYS"),
        ("open chrome",    "CHR"),
        ("open edge",      "EDG"),
        ("open notepad",   "NTP"),
        ("search ",        "SRC"),
    ]
    qf = tk.Frame(left, bg=BG_DEEP)
    qf.pack(fill="x", pady=4)
```

```
    def send_quick(cmd):
        threading.Thread(
            target=execute_manual_command,
            args=(cmd,), daemon=True).start()

    for cmd, label in quick_cmds:
        tk.Button(qf, text=label,
                  command=lambda c=cmd: send_quick(c),
                  bg=BG_PANEL, fg=CYAN,
                  font=(FT, 9, "bold"),
                  relief="flat", width=5,
                  cursor="hand2",
                  activebackground=CYAN_DIM,
                  activeforeground=WHITE
                  ).pack(side="left", padx=2, pady=2)

    # Right panel — Log
    right = tk.Frame(main_f, bg=BG_DEEP)
    right.pack(side="right", fill="both", expand=True)

    tk.Label(right, text="◈  COMMAND LOG",
             font=(FT, 8), fg=TEXT_DIM,
             bg=BG_DEEP).pack(anchor="w", pady=(4,0))

    log_frame = tk.Frame(right, bg=BG_MID,
                         highlightthickness=1,
                         highlightbackground=CYAN_DIM)
    log_frame.pack(fill="both", expand=True)

    log_widget = scrolledtext.ScrolledText(
        log_frame, bg=BG_MID, fg=TEXT_LOG,
        font=(FT, 11), state="disabled",
        bd=0, highlightthickness=0,
        selectbackground=BG_PANEL,
        insertbackground=CYAN,
        spacing3=4
    )
    log_widget.pack(fill="both", expand=True, padx=6, pady=6)
    log_widget.tag_config("command",  foreground=CYAN)
    log_widget.tag_config("response", foreground=GREEN)
    log_widget.tag_config("system",   foreground=GOLD)
    log_widget.tag_config("error",    foreground=RED)
    log_widget.tag_config("image",    foreground=ORANGE)
    log_widget.tag_config("dim",      foreground=TEXT_DIM)

    # ── INPUT BAR ──────────────────────────────────────
    inp_frame = tk.Frame(gui_root, bg=BG_MID,
                         highlightthickness=1,
                         highlightbackground=CYAN_DIM)
    inp_frame.pack(fill="x", padx=8, pady=4)

    tk.Label(inp_frame, text="▶",
             font=(FT, 11, "bold"),
             fg=CYAN, bg=BG_MID).pack(side="left", padx=(8,4))

    entry_var = tk.StringVar()
    entry = tk.Entry(inp_frame, textvariable=entry_var,
                     font=(FT, 12),
                     bg=BG_ENTRY, fg=CYAN,
                     insertbackground=CYAN,
                     relief="flat", bd=0,
                     highlightthickness=0)
    entry.pack(side="left", fill="both", expand=True,
               pady=8, padx=4)
    entry.bind("<Return>", on_manual_send)

    tk.Label(inp_frame, text="TYPE A COMMAND OR USE VOICE",
             font=(FT, 7), fg=TEXT_DIM,
             bg=BG_MID).pack(side="left", padx=4)

    tk.Button(inp_frame, text="SEND ▶",
              command=on_manual_send,
              bg=CYAN_DIM, fg=WHITE,
              font=(FT, 10, "bold"),
```

```
            relief="flat", padx=10,
            cursor="hand2",
            activebackground=CYAN,
            activeforeground=BG_DEEP
            ).pack(side="right", padx=6, pady=6)

    # ── BOTTOM BUTTONS ──────────────────────────────
    btn_f = tk.Frame(gui_root, bg=BG_DEEP)
    btn_f.pack(fill="x", padx=8, pady=(0,8))

    tk.Button(btn_f,
              text="◈  ACTIVATE VOICE  ( Ctrl + Space )",
              command=lambda: threading.Thread(
                  target=manual_activation, daemon=True).start(),
              bg=BG_PANEL, fg=GREEN,
              font=(FT, 11, "bold"),
              relief="flat", pady=10,
              cursor="hand2",
              activebackground=GREEN_DIM,
              activeforeground=WHITE
              ).pack(side="left", expand=True,
                     fill="x", padx=(0,4))

    tk.Button(btn_f,
              text="◈  SHUTDOWN CREATOR",
              command=do_shutdown,
              bg=BG_PANEL, fg=RED,
              font=(FT, 11, "bold"),
              relief="flat", pady=10,
              cursor="hand2",
              activebackground="#330011",
              activeforeground=WHITE
              ).pack(side="right", expand=True,
                     fill="x", padx=(4,0))

    # ── BOTTOM TICKER ───────────────────────────────
    tick_f = tk.Frame(gui_root, bg=BG_MID, height=20)
    tick_f.pack(fill="x")
    tick_f.pack_propagate(False)
    ticker_var = tk.StringVar(
        value="  CREATOR v4.0  ◈  VOICE READY  ◈  "
              "MANUAL CONTROL ACTIVE  ◈  ffmpeg 8.1.1  ◈  "
              "Whisper BASE Model  ◈  Python 3.9.11  ◈  ")
    tk.Label(tick_f, textvariable=ticker_var,
             font=(FT, 7), fg=TEXT_DIM,
             bg=BG_MID, anchor="w").pack(fill="x", padx=8)

    # ── EVENTS ──────────────────────────────────────
    gui_root.protocol("WM_DELETE_WINDOW", do_shutdown)

    # Start animations and threads
    gui_ready.set()
    gui_root.after(100, animate_reactor)
    gui_root.after(500, update_stats)
    gui_root.after(100, draw_top)

    threading.Thread(target=main_loop,       daemon=True).start()
    threading.Thread(target=shortcut_thread, daemon=True).start()

    gui_root.mainloop()

def log_gui(text, tag="system"):
    def _do():
        if not log_widget: return
        ts = datetime.now().strftime("%H:%M:%S")
        log_widget.configure(state="normal")
        log_widget.insert("end", f"[{ts}] ", "dim")
        log_widget.insert("end", f"{text}\n", tag)
        log_widget.configure(state="disabled")
        log_widget.see("end")
    if gui_root:
        try: gui_root.after(0, _do)
        except: pass

def set_mode_display(is_active):
```

```
    def _do():
        if mode_var:
            if is_active:
                mode_var.set("◉  ACTIVE")
            else:
                mode_var.set("◉  STANDBY")
    if gui_root:
        try: gui_root.after(0, _do)
        except: pass

def do_shutdown():
    app.shutdown.set()

def on_manual_send(event=None):
    """Handle manual text input — runs command from text box."""
    if not entry_var: return
    cmd = entry_var.get().strip()
    if not cmd: return
    entry_var.set("")
    threading.Thread(
        target=execute_manual_command,
        args=(cmd,), daemon=True).start()

def execute_manual_command(cmd):
    """Execute a command typed manually — same as voice."""
    log_gui(f"YOU (text): {cmd}", "command")
    if is_end_command(cmd):
        handle_sleep(); return
    route_intent(cmd)

# ═══════════════════════════════════════════════════════════
# SECTION 8 — POPUP ANIMATIONS (Jarvis Style)
# ═══════════════════════════════════════════════════════════
def show_yes_master_popup():
    def _run():
        r = tk.Tk()
        r.title("")
        r.geometry("520x300")
        r.configure(bg=BG_DEEP)
        r.overrideredirect(True)
        r.attributes("-topmost", True)
        try: r.attributes("-alpha", 0.96)
        except: pass

        sw, sh = r.winfo_screenwidth(), r.winfo_screenheight()
        r.geometry(f"520x300+{(sw-520)//2}+{(sh-300)//2}")

        c = tk.Canvas(r, width=520, height=300,
                      bg=BG_DEEP, highlightthickness=0)
        c.pack(fill="both", expand=True)

        # Border
        c.create_rectangle(1,1,519,299, outline=CYAN, width=1)
        c.create_rectangle(4,4,516,296, outline=CYAN_DIM, width=1)

        # HUD corners
        for x1,y1,x2,y2,x3,y3 in [
            (1,28,1,1,28,1),   (492,1,519,1,519,28),
            (1,272,1,299,28,299),(492,299,519,299,519,272)
        ]:
            c.create_line(x1,y1,x2,y2, fill=CYAN, width=2)
            c.create_line(x2,y2,x3,y3, fill=CYAN, width=2)

        cx, cy = 260, 148

        # Scan line
        scan = c.create_line(4, 0, 516, 0,
                             fill=CYAN_DIM, width=1)
        scan_y = [10]

        # Title
        c.create_text(cx, cy-30,
                      text="YES  SIR",
                      font=(FT, 34, "bold"), fill=CYAN,
                      tags="title")
```

```
        c.create_text(cx, cy+14,
                      text="◆  C R E A T O R  O N L I N E  ◆",
                      font=(FT, 11), fill=TEXT_LOG)
        c.create_text(cx, cy+42,
                      text="COMMAND INTERFACE READY",
                      font=(FT, 8), fill=TEXT_DIM)

        # Expanding rings
        rings = []
        ring_data = [
            {"r":30,"speed":2.5,"color":CYAN,"width":2},
            {"r":55,"speed":1.8,"color":CYAN_DIM,"width":1},
            {"r":80,"speed":1.2,"color":TEXT_DIM,"width":1},
        ]
        for rd in ring_data:
            rid = c.create_oval(
                cx-rd["r"], cy-rd["r"],
                cx+rd["r"], cy+rd["r"],
                outline=rd["color"],
                width=rd["width"])
            rings.append({"id":rid, "r":rd["r"],
                          "speed":rd["speed"],
                          "max":rd["r"]+100})

        # Scanner arc
        arc = c.create_arc(cx-90, cy-90, cx+90, cy+90,
                           start=0, extent=80,
                           outline=GREEN, width=2,
                           style="arc")
        arc_a = [0]
        title_colors = [CYAN,"#00BBEE","#00EEFF",
                        "#00BBEE",CYAN]
        frame = [0]

        def animate():
            frame[0] += 1

            # Rotate scanner
            arc_a[0] = (arc_a[0] + 12) % 360
            c.itemconfig(arc, start=arc_a[0])

            # Pulse title
            c.itemconfig("title",
                fill=title_colors[frame[0] % len(title_colors)])

            # Expand rings
            for ring in rings:
                ring["r"] += ring["speed"]
                if ring["r"] > ring["max"]:
                    ring["r"] = 20
                rr = ring["r"]
                c.coords(ring["id"],
                         cx-rr, cy-rr, cx+rr, cy+rr)

            # Scan line
            scan_y[0] += 4
            if scan_y[0] > 296: scan_y[0] = 4
            c.coords(scan, 4, scan_y[0], 516, scan_y[0])

            r.after(40, animate)

        animate()
        r.after(2800, r.destroy)
        r.mainloop()

    threading.Thread(target=_run, daemon=True).start()


def show_goodbye_popup():
    def _run():
        r = tk.Tk()
        r.title("")
        r.geometry("520x300")
        r.configure(bg=BG_DEEP)
        r.overrideredirect(True)
```

```
        r.attributes("-topmost", True)
        try: r.attributes("-alpha", 0.96)
        except: pass

        sw, sh = r.winfo_screenwidth(), r.winfo_screenheight()
        r.geometry(f"520x300+{(sw-520)//2}+{(sh-300)//2}")

        c = tk.Canvas(r, width=520, height=300,
                      bg=BG_DEEP, highlightthickness=0)
        c.pack(fill="both", expand=True)

        c.create_rectangle(1,1,519,299, outline=RED, width=1)
        c.create_rectangle(4,4,516,296, outline="#660022", width=1)

        for x1,y1,x2,y2,x3,y3 in [
            (1,28,1,1,28,1),   (492,1,519,1,519,28),
            (1,272,1,299,28,299),(492,299,519,299,519,272)
        ]:
            c.create_line(x1,y1,x2,y2, fill=RED, width=2)
            c.create_line(x2,y2,x3,y3, fill=RED, width=2)

        cx, cy = 260, 148

        title = c.create_text(cx, cy-28,
                              text="GOODBYE  SIR",
                              font=(FT, 30, "bold"), fill=RED)
        c.create_text(cx, cy+14,
                      text="◆  S Y S T E M  O F F L I N E  ◆",
                      font=(FT, 11), fill="#AA2233")
        c.create_text(cx, cy+42,
                      text="STANDING BY — READY TO REACTIVATE",
                      font=(FT, 8), fill=TEXT_DIM)

        # Contracting rings
        rings = []
        ring_data = [
            {"r":120,"speed":2.8,"color":RED,"width":2},
            {"r":90, "speed":2.0,"color":"#660022","width":1},
            {"r":60, "speed":1.4,"color":"#330011","width":1},
        ]
        for rd in ring_data:
            rid = c.create_oval(
                cx-rd["r"], cy-rd["r"],
                cx+rd["r"], cy+rd["r"],
                outline=rd["color"], width=rd["width"])
            rings.append({"id":rid, "r":rd["r"],
                          "speed":rd["speed"]})

        # Fading arc
        arc   = c.create_arc(cx-85, cy-85, cx+85, cy+85,
                             start=0, extent=80,
                             outline=RED, width=2, style="arc")
        arc_a = [0]
        fade_colors = [RED,"#CC2244","#992233",
                       "#661122","#330011"]
        frame = [0]

        def animate():
            frame[0] += 1
            arc_a[0] = (arc_a[0] - 10) % 360
            c.itemconfig(arc, start=arc_a[0])

            fc = fade_colors[min(frame[0]//8,
                                 len(fade_colors)-1)]
            c.itemconfig(title, fill=fc)

            for ring in rings:
                ring["r"] -= ring["speed"]
                if ring["r"] < 5: ring["r"] = 130
                rr = ring["r"]
                c.coords(ring["id"],
                         cx-rr, cy-rr, cx+rr, cy+rr)
            r.after(50, animate)

        animate()
```

```
        r.after(2800, r.destroy)
        r.mainloop()

    threading.Thread(target=_run, daemon=True).start()


def show_cancel_button():
    def _run():
        g = tk.Tk()
        g.title("")
        g.geometry("280x120")
        g.configure(bg=BG_DEEP)
        g.overrideredirect(True)
        g.attributes("-topmost", True)

        # Position bottom-right corner
        sw = g.winfo_screenwidth()
        sh = g.winfo_screenheight()
        g.geometry(f"280x120+{sw-296}+{sh-160}")

        c = tk.Canvas(g, width=280, height=120,
                      bg=BG_DEEP, highlightthickness=0)
        c.pack(fill="both", expand=True)
        c.create_rectangle(1,1,279,119,
                           outline=CYAN_DIM, width=1)
        c.create_text(140, 28,
                      text="◆  LISTENING",
                      font=(FT, 11, "bold"), fill=CYAN)

        def on_cancel():
            app.cancel.set()
            speak("Cancelled.")
            try: g.destroy()
            except: pass

        btn = tk.Button(g, text="⛔  CANCEL  TASK",
                        command=on_cancel,
                        bg="#1A0008", fg=RED,
                        font=(FT, 12, "bold"),
                        relief="flat", cursor="hand2",
                        activebackground="#330011",
                        activeforeground=WHITE)
        btn.place(x=14, y=52, width=252, height=52)

        def auto_close():
            time.sleep(LISTEN_DURATION + 2)
            try: g.destroy()
            except: pass

        threading.Thread(target=auto_close, daemon=True).start()
        g.mainloop()

    threading.Thread(target=_run, daemon=True).start()

# ═══════════════════════════════════════════════════════════════
# SECTION 9 — PERMISSION + HELPERS
# ═══════════════════════════════════════════════════════════════
def ask_permission(action_text):
    speak(f"Do you want me to {action_text}? Say yes or no.")
    ok   = record_audio(3)
    resp = transcribe_audio() if ok else ""
    return "yes" in (resp.lower() if resp else "")

def is_cancel(text):
    return any(w in (text or "").lower() for w in CANCEL_WORDS)

def is_end_command(text):
    t = (text or "").lower()
    return any(e in t for e in END_WORDS)

# ═══════════════════════════════════════════════════════════════
# SECTION 10 — MULTI-COMMAND PARSER
# ═══════════════════════════════════════════════════════════════
def split_commands(text):
    seps = [" and then "," then "," and also "," also "," and "]
```

```
    parts = [text]
    for sep in seps:
        new = []
        for p in parts: new.extend(p.split(sep))
        parts = new
    return [p.strip() for p in parts if p.strip()]

# ═══════════════════════════════════════════════════════════════
# SECTION 11 — AI BRAIN
# ═══════════════════════════════════════════════════════════════
def ask_ai(prompt):
    if not AI_AVAILABLE:
        return ("AI mode not enabled. "
                "I can open apps, search, and control your PC.")
    try:
        resp = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[
                {"role":"system",
                 "content":"You are CREATOR, a Jarvis-like AI. "
                           "Be concise — max 2 sentences."},
                {"role":"user","content":prompt}
            ],
            max_tokens=150
        )
        return resp.choices[0].message.content
    except Exception as e:
        return f"AI Error: {e}"

# ═══════════════════════════════════════════════════════════════
# SECTION 12 — IMAGE GENERATION
# ═══════════════════════════════════════════════════════════════
def generate_image(prompt):
    if not AI_AVAILABLE:
        speak("Image generation needs an OpenAI API key.")
        return
    speak(f"Generating image of {prompt}. Please wait.")
    log_gui(f"Generating image: {prompt}", "image")
    try:
        resp = client.images.generate(
            model="dall-e-3", prompt=prompt,
            size="1024x1024", n=1)
        url  = resp.data[0].url
        desk = os.path.join(os.path.expanduser("~"), "Desktop")
        name = re.sub(r"[^a-zA-Z0-9_]", "_", prompt[:30])
        path = os.path.join(desk, f"CREATOR_{name}.png")
        urllib.request.urlretrieve(url, path)
        speak("Image saved to your desktop.")
        log_gui(f"Image saved: {path}", "image")
        subprocess.Popen(["explorer", path])
    except Exception as e:
        speak(f"Image generation failed.")
        log_gui(f"Image error: {e}", "error")

# ═══════════════════════════════════════════════════════════════
# SECTION 13 — MASTER FILE CONTROL
# ═══════════════════════════════════════════════════════════════
def master_file_control(text):
    t = text.lower()

    m = re.search(r"create (?:a )?file ([^\s]+)", t)
    if m:
        fn = m.group(1)
        cm = re.search(r"with (?:content|text) (.+)", text, re.I)
        ct = cm.group(1) if cm else ""
        if ask_permission(f"create file {fn}"):
            try:
                with open(fn,"w") as f: f.write(ct)
                speak(f"File {fn} created.")
                log_gui(f"Created: {fn}", "system")
            except Exception as e: speak(f"Error: {e}")
        else: speak("Cancelled.")
        return True

    m = re.search(r"read (?:the )?file ([^\s]+)", t)
```

```
    if m:
        fn = m.group(1)
        try:
            with open(fn,"r") as f: ct = f.read()
            speak(f"{fn} contains: {ct[:300]}" if ct
                  else f"{fn} is empty.")
        except Exception as e: speak(f"Cannot read: {e}")
        return True

    m = re.search(r"delete (?:the )?file ([^\s]+)", t)
    if m:
        fn = m.group(1)
        if ask_permission(f"delete file {fn}"):
            try:
                os.remove(fn)
                speak(f"File {fn} deleted.")
                log_gui(f"Deleted file: {fn}", "system")
            except Exception as e: speak(f"Error: {e}")
        else: speak("Cancelled.")
        return True

    m = re.search(r"(?:add|append) (?:text|content) (.+?) to ([^\s]+)",
                  text, re.I)
    if m:
        ct,fn = m.group(1),m.group(2)
        if ask_permission(f"add text to {fn}"):
            try:
                with open(fn,"a") as f: f.write("\n"+ct)
                speak(f"Text added to {fn}.")
            except Exception as e: speak(f"Error: {e}")
        else: speak("Cancelled.")
        return True

    m = re.search(r"rewrite ([^\s]+) with (?:content|text) (.+)",
                  text, re.I)
    if m:
        fn,ct = m.group(1),m.group(2)
        if ask_permission(f"rewrite {fn}"):
            try:
                with open(fn,"w") as f: f.write(ct)
                speak(f"{fn} rewritten.")
            except Exception as e: speak(f"Error: {e}")
        else: speak("Cancelled.")
        return True

    m = re.search(r"rename (?:file )?([^\s]+) to ([^\s]+)", t)
    if m:
        old,new = m.group(1),m.group(2)
        if ask_permission(f"rename {old} to {new}"):
            try:
                os.rename(old,new)
                speak(f"Renamed to {new}.")
            except Exception as e: speak(f"Error: {e}")
        else: speak("Cancelled.")
        return True

    m = re.search(r"create (?:a )?(?:folder|directory) ([^\s]+)", t)
    if m:
        fn = m.group(1)
        if ask_permission(f"create folder {fn}"):
            try:
                os.makedirs(fn, exist_ok=True)
                speak(f"Folder {fn} created.")
                log_gui(f"Folder created: {fn}", "system")
            except Exception as e: speak(f"Error: {e}")
        else: speak("Cancelled.")
        return True

    m = re.search(r"delete (?:the )?(?:folder|directory) ([^\s]+)", t)
    if m:
        fn = m.group(1)
        if ask_permission(f"delete folder {fn} and all contents"):
            try:
                shutil.rmtree(fn)
                speak(f"Folder {fn} deleted.")
```

```
                log_gui(f"Folder deleted: {fn}", "system")
            except Exception as e: speak(f"Error: {e}")
        else: speak("Cancelled.")
        return True

    m = re.search(r"list (?:files|contents)(?: in)? ?([^\s]+)?", t)
    if m:
        path = m.group(1) or "."
        try:
            items = os.listdir(os.path.expandvars(path))
            if items:
                speak(f"Found {len(items)} items: "
                      + ", ".join(items[:8]))
            else: speak(f"{path} is empty.")
            log_gui(f"Listed: {path}", "system")
        except Exception as e: speak(f"Cannot list: {e}")
        return True

    return False

# ═══════════════════════════════════════════════════════════
# SECTION 14 — EDGE AUTOMATION
# ═══════════════════════════════════════════════════════════
def open_edge_task(text):
    t    = text.lower()
    gom  = re.search(r"(?:go to|navigate to|visit) ([\w./-]+)", t)
    srm  = re.search(
        r"search (?:for |on edge for |on edge )?(.+?)(?:\s+on edge)?$", t)

    if gom:
        site = gom.group(1)
        if not site.startswith("http"): site = "https://"+site
        if ask_permission(f"open Edge and go to {site}"):
            subprocess.Popen([EDGE_PATH, site])
            speak(f"Opening Edge on {site}.")
        else: speak("Cancelled.")
    elif srm and "edge" in t:
        q   = srm.group(1).strip()
        url = f"https://www.bing.com/search?q={q.replace(' ','+')}"
        if ask_permission(f"search for {q} on Edge"):
            subprocess.Popen([EDGE_PATH, url])
            speak(f"Searching Edge for {q}.")
        else: speak("Cancelled.")
    else:
        if ask_permission("open Microsoft Edge"):
            subprocess.Popen([EDGE_PATH])
            speak("Opening Edge.")
        else: speak("Cancelled.")

# ═══════════════════════════════════════════════════════════
# SECTION 15 — OS CONTROLS
# ═══════════════════════════════════════════════════════════
def change_brightness(pct):
    pct = max(0, min(100, int(pct)))
    if ask_permission(f"set brightness to {pct}%"):
        cmd = (f"powershell (Get-WmiObject -Namespace root/WMI "
               f"-Class WmiMonitorBrightnessMethods)"
               f".WmiSetBrightness(1,{pct})")
        subprocess.run(cmd, shell=True)
        speak(f"Brightness set to {pct}%.")
    else: speak("Cancelled.")

def toggle_wifi(on=True):
    act = "enable" if on else "disable"
    if ask_permission(f"{act} Wi-Fi"):
        ac = "Enable-NetAdapter" if on else "Disable-NetAdapter"
        subprocess.run(
            f"powershell Get-NetAdapter -Name *Wi-Fi* "
            f"| {ac} -Confirm:$false", shell=True)
        speak(f"Wi-Fi {act}d.")
    else: speak("Cancelled.")

def toggle_bluetooth(on=True):
    st = "on" if on else "off"
    if ask_permission(f"turn Bluetooth {st}"):
```

```
        sc = "Start-Service bthserv" if on else "Stop-Service bthserv"
        subprocess.run(
            f"powershell Set-Service -Name bthserv "
            f"-StartupType Automatic; {sc}", shell=True)
        speak(f"Bluetooth {st}.")
    else: speak("Cancelled.")

def system_power(action):
    if ask_permission(f"{action} the system"):
        speak(f"{action.capitalize()} in 3 seconds.")
        cmds = {
            "shutdown":"shutdown /s /t 3",
            "restart":"shutdown /r /t 3",
            "sleep":"rundll32.exe powrprof.dll,SetSuspendState 0,1,0"
        }
        subprocess.run(cmds[action], shell=True)
    else: speak("Cancelled.")

def get_system_info():
    cpu  = psutil.cpu_percent(interval=0.5)
    ram  = psutil.virtual_memory().percent
    bat  = psutil.sensors_battery()
    batt = f"{int(bat.percent)}%" if bat else "N/A"
    speak(f"CPU {cpu}%, RAM {ram}%, Battery {batt}.")

# ═══════════════════════════════════════════════════════════════
# SECTION 16 — APP MAPS
# ═══════════════════════════════════════════════════════════════
WEB_APPS = {
    "youtube":"https://www.youtube.com",
    "gmail":"https://mail.google.com",
    "spotify":"https://open.spotify.com",
    "twitter":"https://www.twitter.com",
    "facebook":"https://www.facebook.com",
    "github":"https://www.github.com",
    "netflix":"https://www.netflix.com",
    "google":"https://www.google.com",
}
DESKTOP_APPS = {
    "whatsapp":  "powershell Start-Process shell:AppsFolder\\
5319275A.WhatsAppDesktop_cv1g1gvanyjgm!App",
    "instagram": "powershell Start-Process shell:AppsFolder\\Facebook.InstagramBeta_8xx8rvfyw5nnt!
App",
    "notepad":"notepad.exe",
    "calculator":"calc.exe",
    "paint":"mspaint.exe",
    "task manager":"taskmgr.exe",
    "file explorer":"explorer.exe",
    "cmd":"cmd.exe",
    "chrome":r"C:\Program Files\Google\Chrome\Application\chrome.exe",
    "brave":r"C:\Program Files\BraveSoftware\Brave-Browser\Application\brave.exe",
    "vscode":r"C:\Users\%USERNAME%\AppData\Local\Programs\Microsoft VS Code\Code.exe",
}

# ═══════════════════════════════════════════════════════════════
# SECTION 17 — SINGLE COMMAND ROUTER
# ═══════════════════════════════════════════════════════════════
def route_single(text):
    t = (text or "").lower().strip()
    if not t: return
    if is_cancel(t):
        app.cancel.set(); speak("Cancelled. / रद्द किया।"); return

    # ── WHATSAPP MESSAGE ──────────────────────────────────
    m = re.search(
        r"(?:send|message|whatsapp) (?:message to |to )?(.+?) (?:saying |say |message |with message
)?[\"']?(.+)[\"']?",
        t, re.IGNORECASE)
    if m and ("whatsapp" in t or "message" in t or "send" in t):
        send_whatsapp_message(m.group(1).strip(), m.group(2).strip())
        return

    # ── SMART SEARCH ──────────────────────────────────────
    m = re.search(r"(?:what is|who is|how to|why is|when is|where is|tell me about|search) (.+)",
t)
```

```
    if m:
        smart_search(m.group(1)); return
    hindi_map = {
        "क्रोम खोलो":"open chrome","chrome खोलो":"open chrome",
        "नोटपैड खोलो":"open notepad","notepad खोलो":"open notepad",
        "यूटयूब खोलो":"open youtube","youtube खोलो":"open youtube",
        "इंस्टाग्राम खोलो":"open instagram",
        "कैलकुलेटर खोलो":"open calculator",
        "एज खोलो":"open edge","edge खोलो":"open edge",
        "व्हाट्सएप खोलो":"open whatsapp",
        "जीमेल खोलो":"open gmail",
        "बंद करो":"bye creator","बाय":"bye creator",
        "सिस्टम जानकारी":"system info",
        "बंद करो कंप्यूटर":"shutdown",
        "रीस्टार्ट करो":"restart",
    }
    for hindi, english in hindi_map.items():
        if hindi in text:
            t = english
            break

    m = re.search(
        r"(?:generate|create|make) (?:an? )?image "
        r"(?:of|showing|with) (.+)", t)
    if m: generate_image(m.group(1)); return

    if master_file_control(text): return

    if "edge" in t: open_edge_task(text); return

    m = re.search(r"set brightness to (\d+)", t)
    if m: change_brightness(m.group(1)); return

    m = re.search(r"(turn|toggle) (wifi|wi-fi) (on|off)", t)
    if m: toggle_wifi(m.group(3)=="on"); return

    m = re.search(r"(turn|toggle) bluetooth (on|off)", t)
    if m: toggle_bluetooth(m.group(2)=="on"); return

    if "shutdown" in t: system_power("shutdown"); return
    if "restart" in t or "reboot" in t:
        system_power("restart"); return
    if "standby" in t: system_power("sleep"); return

    if any(x in t for x in
           ["system info","status","cpu","ram","battery"]):
        get_system_info(); return

    for name, url in WEB_APPS.items():
        if f"open {name}" in t:
            if ask_permission(f"open {name}"):
                webbrowser.open(url)
                speak(f"Opening {name}.")
            else: speak("Cancelled.")
            return

    for name, path in DESKTOP_APPS.items():
        if f"open {name}" in t:
            if ask_permission(f"open {name}"):
                try:
                    if path.startswith("powershell"):
                        subprocess.Popen(path, shell=True)
                    elif path.startswith("shell:"):
                        subprocess.Popen(["explorer", path])
                    else:
                        subprocess.Popen(os.path.expandvars(path))
                    speak(f"Opening {name}.")
                except: speak(f"Could not open {name}.")
            else: speak("Cancelled.")
            return

    m = re.search(r"(search|google) (.+)", t)
    if m:
        q = m.group(2)
        if ask_permission(f"search for {q}"):
```

```
            webbrowser.open(
                "https://www.google.com/search?q="
                + q.replace(" ","+"))
            speak(f"Searching for {q}.")
        else: speak("Cancelled.")
        return

    speak(ask_ai(text))

# ═══════════════════════════════════════════════════════════════
# SECTION 18 — MULTI-COMMAND ROUTER
# ═══════════════════════════════════════════════════════════════
def route_intent(text):
    if is_end_command(text): handle_sleep(); return
    commands = split_commands(text)
    log_gui(f"YOU: {text}", "command")
    if len(commands) > 1:
        speak(f"Running {len(commands)} commands.")
        for i, cmd in enumerate(commands, 1):
            if app.cancel.is_set() or app.shutdown.is_set(): break
            if is_end_command(cmd): handle_sleep(); return
            log_gui(f"Step {i}: {cmd}", "command")
            route_single(cmd)
            time.sleep(0.4)
    else:
        route_single(text)

# ═══════════════════════════════════════════════════════════════
# SECTION 19 — SLEEP / SHUTDOWN
# ═══════════════════════════════════════════════════════════════
def handle_sleep():
    show_goodbye_popup()
    speak("Goodbye Sir. I am going to sleep.")
    app.set_mode(STANDBY)
    set_mode_display(False)
    log_gui("CREATOR went to sleep.", "system")

def handle_full_shutdown():
    show_goodbye_popup()
    speak("Goodbye Sir. Shutting down completely.")
    log_gui("CREATOR shut down.", "system")
    time.sleep(3)
    sys.exit(0)

# ═══════════════════════════════════════════════════════════════
# SECTION 20 — ACTIVE CONTINUOUS MODE
# ═══════════════════════════════════════════════════════════════
def run_active_mode():
    app.set_mode(ACTIVE)
    set_mode_display(True)
    log_gui("ACTIVE — continuous listening.", "system")

    while app.get_mode()==ACTIVE and not app.shutdown.is_set():
        app.cancel.clear()
        show_cancel_button()
        ok = record_audio(LISTEN_DURATION)
        if not ok:
            speak("Mic error. Please check your microphone.")
            time.sleep(1); continue
        task = transcribe_audio()
        if not task: continue
        print(f"YOU: {task}")
        if is_end_command(task): handle_sleep(); return
        if is_cancel(task):
            speak("Okay, ready for your next command.")
            app.cancel.clear(); continue
        app.cancel.clear()
        route_intent(task)
    app.set_mode(STANDBY)

# ═══════════════════════════════════════════════════════════════
# SECTION 21 — WAKE SEQUENCE
# ═══════════════════════════════════════════════════════════════
def trigger_wake():
    if app.get_mode() == ACTIVE: return
```

```
    show_yes_master_popup()
    speak("Yes Sir. I am awake.")
    set_mode_display(True)
    log_gui("Wake word detected → ACTIVE mode.", "system")
    run_active_mode()

def manual_activation():
    if app.get_mode() == ACTIVE:
        speak("I am already active, Sir."); return
    trigger_wake()

# ═══════════════════════════════════════════════════════════════
# SECTION 22 — MAIN STANDBY LOOP
# ═══════════════════════════════════════════════════════════════
def main_loop():
    """
    PUSH TO LISTEN mode:
    - Does NOT record mic continuously
    - Only activates when Ctrl+Space pressed OR wake word button clicked
    - Saves CPU, RAM, and battery significantly
    - Wake word via keyboard.wait() only
    """
    gui_ready.wait()
    log_gui("CREATOR online. Press Ctrl+Space or click ACTIVATE.", "system")
    speak("CREATOR is ready and standing by.")
    while not app.shutdown.is_set():
        time.sleep(0.5)  # Just wait — no continuous mic polling
    handle_full_shutdown()

# ═══════════════════════════════════════════════════════════════
# SECTION 23 — SHORTCUT THREAD
# ═══════════════════════════════════════════════════════════════
def shortcut_thread():
    """
    Handles BOTH Ctrl+Space AND wake word detection.
    After Ctrl+Space: listens once for wake word confirmation.
    After wake word heard: activates CREATOR.
    """
    gui_ready.wait()
    while not app.shutdown.is_set():
        try:
            keyboard.wait(SHORTCUT_KEY)
            if app.get_mode() == ACTIVE:
                speak("I am already active, Sir."); continue
            # Listen for wake word
            speak("Listening.")
            ok = record_audio(WAKE_CHECK_DURATION)
            if not ok: continue
            heard = transcribe_audio()
            if heard and WAKE_WORD in heard.lower():
                threading.Thread(
                    target=trigger_wake, daemon=True).start()
            else:
                log_gui("Wake word not detected.", "dim")
        except Exception as e:
            print(f"[HOTKEY] {e}"); time.sleep(0.5)

# ═══════════════════════════════════════════════════════════════
# SECTION 24 — ENTRY POINT
# ═══════════════════════════════════════════════════════════════
if __name__ == "__main__":
    print("=" * 55)
    print("     CREATOR AI ASSISTANT — VERSION 4.0")
    print("=" * 55)
    print(f"  AI Mode  : {'Enabled' if AI_AVAILABLE else 'Disabled'}")
    print(f"  Model    : Whisper {MODEL_SIZE} (faster)")
    print("=" * 55)
    build_gui()
```